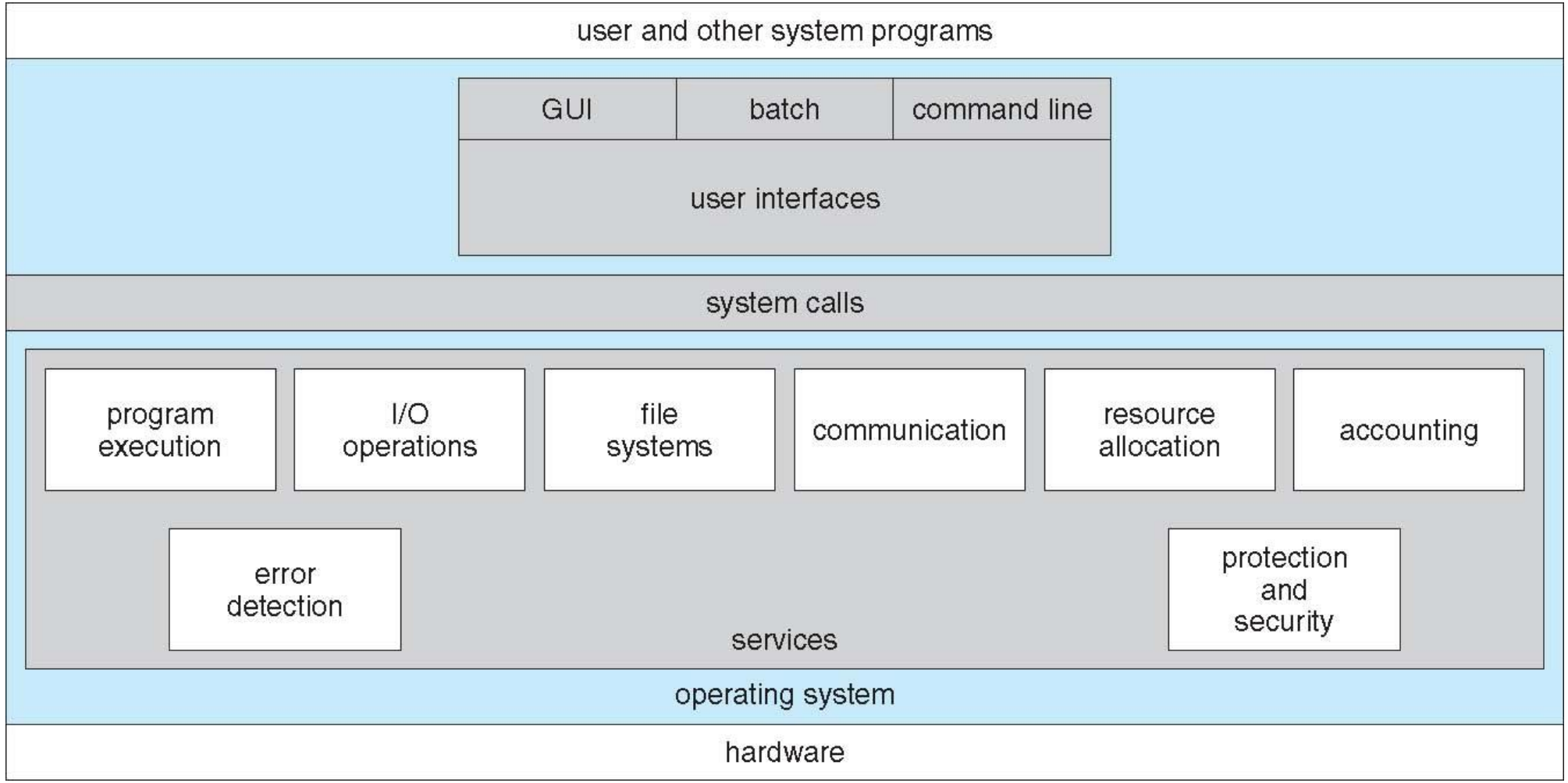
# OPERATING SYSTEMS
## (BIT-202)

2

# OPERATING-SYSTEM STRUCTURES

# A VIEW OF OPERATING SYSTEM SERVICES



# OPERATING SYSTEM SERVICES

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (UI).
    - Varies between Command-Line (CLI), Graphics User Interface (GUI), Batch
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

# OPERATING SYSTEM SERVICES (CONT.)

- One set of operating-system services provides functions that are helpful to the user (Cont.):
  - **Communications** – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)
  - **Error detection** – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

# OPERATING SYSTEM SERVICES (CONT.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
  - **Accounting** - To keep track of which users use how much and what kinds of computer resources
  - **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
    - Protection involves ensuring that all access to system resources is controlled
    - Security of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

# OPERATING SYSTEM USER INTERFACE- CLI

- CLI or command interpreter allows direct command entry
- On systems with multiple command interpreters to choose from, the interpreters are known as shells.
  - Example shells in UNIX
- Two ways to execute commands
  - Implemented in kernel only
  - Implemented through system programs

# OPERATING SYSTEM USER INTERFACE- GUI

- Rather than entering commands, users employ a mouse and menu based system.
- User-friendly desktop metaphor interface
  - Usually mouse, keyboard, and monitor
  - Icons represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a folder)
- First GUI by the Xerox in 1973.
- Many systems now include both CLI and GUI interfaces

# OPERATING SYSTEM USER INTERFACE - GUI

- User-friendly desktop metaphor interface
  - Usually mouse, keyboard, and monitor
  - Icons represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a folder)
  - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI "command" shell
  - Apple Mac OS X is "Aqua" GUI interface with UNIX kernel underneath and shells available
  - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

# TOUCHSCREEN INTERFACES

- Touchscreen devices require new interfaces
  - Mouse not possible or not desired
  - Actions and selection based on gestures
  - Virtual keyboard for text entry
- Voice commands.

11

# THREADS & MULTITHREADING

# WHAT IS THREAD ?

- **What is a Thread?**
- A thread is a **single sequence of execution** within a process.
- Threads are also known as **lightweight processes** because they share some properties of processes but are more efficient.

# KEY CHARACTERISTICS OF THREADS

- Belongs to a Process
  - Each thread exists within a single process.
  - A process can have multiple threads.
- Shared Resources
  - Threads within the same process share:
    - Code Section
    - Data Section
    - Operating System Resources (e.g., open files, signals).
- Thread-Specific Resources
  - Each thread has its own:
    - Thread Control Block (TCB)
    - Thread ID
    - Program Counter
    - Register Set
    - Stack

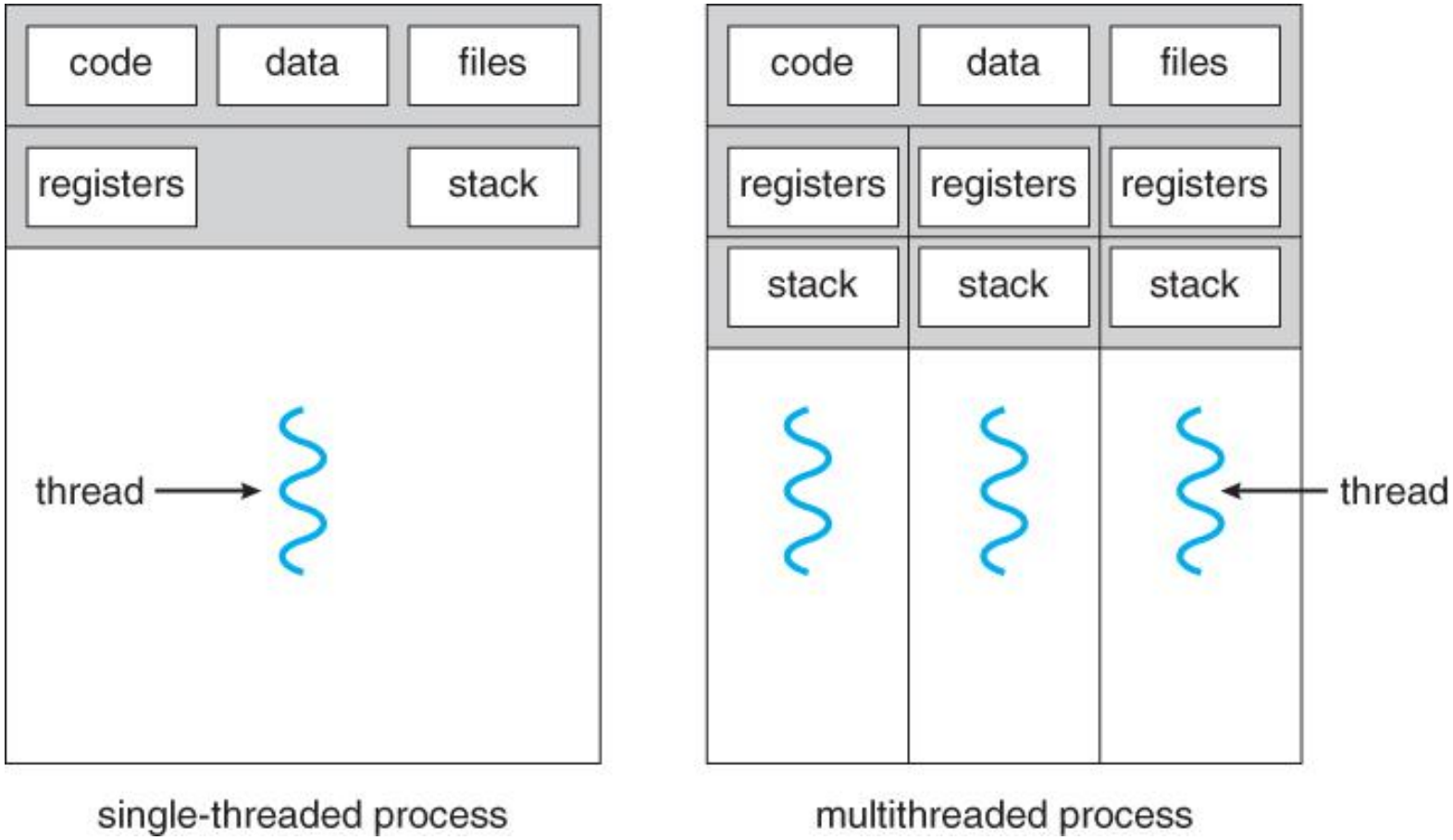
# COMPONENTS OF THREAD

- These are the basic components of the Operating System.
- **Stack Space**: Stores local variables, function calls, and return addresses specific to the thread.
- **Register Set**: Hold temporary data and intermediate results for the thread's execution.
- **Program Counter**: Tracks the current instruction being executed by the thread.

# MULTITHREADING

- Multithreading: The ability of a process to execute multiple threads simultaneously.
- Effectiveness:
  - Requires multiple CPUs or cores for parallel execution.
  - On a single CPU, threads context switch to share CPU time.

# SINGLE THREADED PROCESS AND MULTI-THREADED PROCESS



# ADVANTAGES OF THREADS

- **Improved Performance**: Faster than processes due to shared resources.
- **Efficient Resource Utilization**: Threads share code and data, reducing overhead.
- **Parallelism**: Allows simultaneous execution on multi-core systems.
- **Responsiveness**: Increases application responsiveness (e.g., GUI applications).

# DISADVANTAGES OF THREADS

- **Complexity in Programming:**
  - Writing and debugging multithreaded applications is more challenging.
- **Synchronization Issues:**
  - Shared resources require synchronization to avoid race conditions and deadlocks.
- **Context Switching Overhead:**
  - Frequent context switching can degrade performance, especially on a single CPU.
- **Security Risks:**
  - A bug in one thread can compromise the entire process.
- **Limited Isolation:**
  - Threads lack isolation; if one thread crashes, the entire process may terminate.



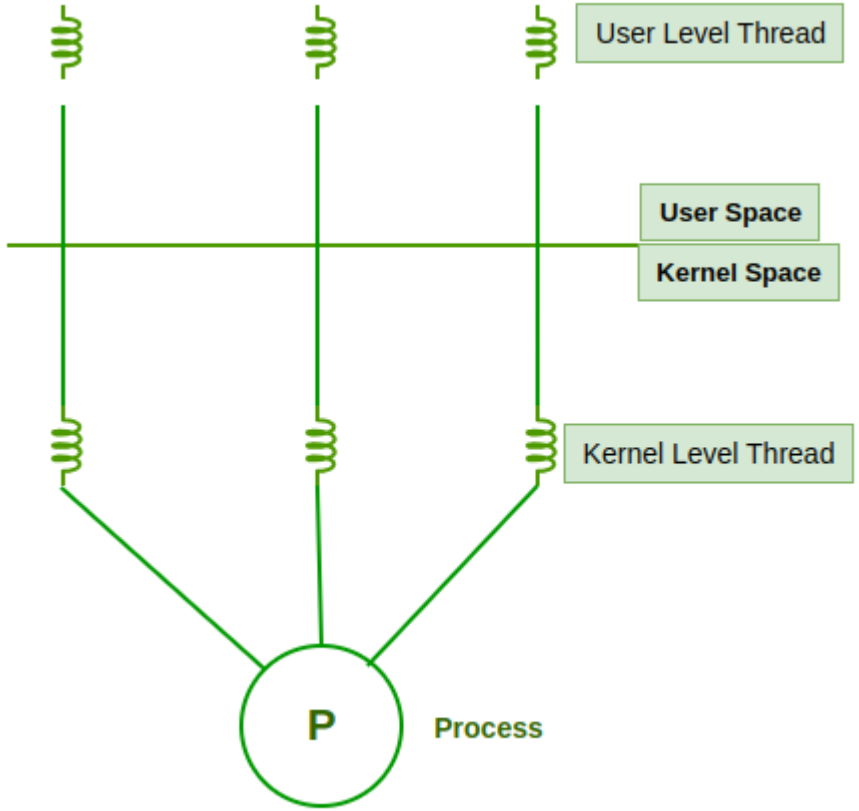
@PRAKHAR

# LIFECYCLE OF A THREAD

- **New:** The lifecycle of a born thread (new thread) starts in this state. It remains in this state till a program starts.
- **Runnable :** A thread becomes runnable after it starts. It is considered to be executing the task given to it.
- **Waiting :** While waiting for another thread to perform a task, the currently running thread goes into the waiting state and then transitions back again after receiving a signal from the other thread.
- **Timed Waiting:** A runnable thread enters into this state for a specific time interval and then transitions back when the time interval expires or the event the thread was waiting for occurs.
- **Terminated (Dead) :** A thread enters into this state after completing its task.

# TYPES OF THREADS

- Threads are implemented in following two ways :
  - User Level Threads (ULT)
  - Kernel Level Threads (KLT)



# USER LEVEL THREAD (ULT)

- User-level threads are threads that are managed entirely by user-space libraries, without direct intervention from the operating system kernel. The kernel treats these threads as part of a single-threaded process.
- User-level threads are created, scheduled, and managed in user space using thread libraries such as POSIX Threads (Pthreads), Java Threads, or Windows User Threads.
- The kernel is unaware of these threads and only recognizes the main process.
- They are lightweight, requiring minimal overhead, and execute faster than kernel-level threads.
- Each thread consists of a Program Counter (PC), Stack, Registers, and a small Process Control Block (PCB).

# ADVANTAGES OF USER-LEVEL THREADS

- **Ease of Implementation**:
  - Can be implemented without requiring changes or support from the operating system.
  - Does not rely on kernel mode or system calls, reducing development complexity.
- **Faster Context Switching**:
  - Switching between threads is handled in user space, avoiding the overhead of switching to kernel mode.
- **Efficiency**:
  - Requires fewer resources and less overhead since the kernel is not involved in their management.
- **Simple Representation**:
  - A thread only consists of essential elements:
    - **Program Counter**: Tracks the next instruction to execute.
    - **Register Set**: Maintains the state of the CPU for the thread.
    - **Stack Space**: Holds the function calls and local variables for the thread.

# DISADVANTAGES OF USER-LEVEL THREADS

- **Lack of Kernel Awareness**:
  - Since the kernel is unaware of the existence of these threads, it cannot provide advanced optimizations like load balancing.
- **Blocking System Calls**:
  - If a thread makes a blocking system call (e.g., waiting for I/O), the **entire process, including all other threads**, is blocked.
- **Complex Scheduling**:
  - Scheduling must be handled by the user-level library or application itself.
  - This adds complexity and may not be as efficient as kernel-level scheduling algorithms.

# KERNEL LEVEL THREADS (KLT)

- Threads that are directly managed by the **operating system kernel**, with the kernel maintaining a thread table to track and control their execution. The kernel is responsible for **scheduling, synchronization, and management** of these threads.
- Kernel-level threads are implemented in both **user space** and **kernel space**.
- Every thread is treated as an **independent entity** by the operating system, with its own execution state and resources.

# ADVANTAGES OF KERNEL-LEVEL THREADS

- **Multicore Support:**
  - Threads can run on multiple processors or cores simultaneously, enabling true parallelism.
- **Kernel Scheduling:**
  - The kernel manages all threads and can use sophisticated scheduling algorithms to optimize performance.
  - It ensures fairness and load balancing across available resources.
- **Blocking Operations:**
  - A blocking system call by one thread does not block the entire process.
  - Other threads can continue to execute independently.
- **Resource Utilization:**
  - The kernel can dynamically distribute threads across CPUs, ensuring efficient use of system resources.

# DISADVANTAGES OF KERNEL-LEVEL THREADS

- **Slower Context Switching:**
  - Context switching involves a transition from user mode to kernel mode and back, adding significant overhead.
- **Higher CPU Overhead:**
  - Managing kernel-level threads requires frequent system calls and kernel interactions, leading to increased resource consumption.
- **Complex Implementation:**
  - Requires substantial support and management from the operating system, making the implementation more intricate.
- **Scheduler Overload:**
  - A large number of threads can overwhelm the kernel scheduler, causing performance degradation in systems with many concurrent threads.

# DIFFERENCE BETWEEN ULT AND KLT

| Aspect | User-Level Threads (ULTs) | Kernel-Level Threads (KLTs) |
|---|---|---|
| **Management** | Managed by the user/application. | Managed by the OS kernel. |
| **Context Switching Time** | Faster (no kernel involvement). | Slower (involves kernel mode). |
| **Blocking Calls** | Entire process blocks. | Other threads can continue. |
| **Implementation** | Easier and simpler. | Complex and resource-intensive. |
| **Multicore Utilization** | Not supported by the kernel. | Fully supported by the kernel. |

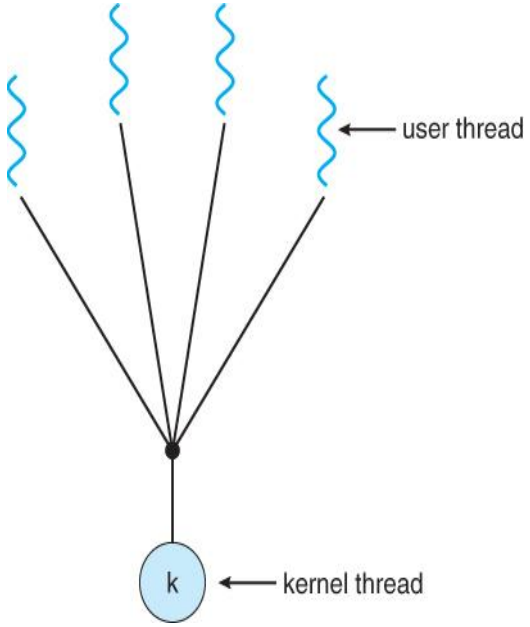
# MULTI-THREADING MODELS

- Some operating system provides a combined user level thread and kernel level thread facility. Solaris is a good example of this combined approch.
- Multi-Threading models are three types.
  - Many-to-One Model
  - One-to-One Model
  - Many-to-Many Model

# MANY-TO-ONE MODEL

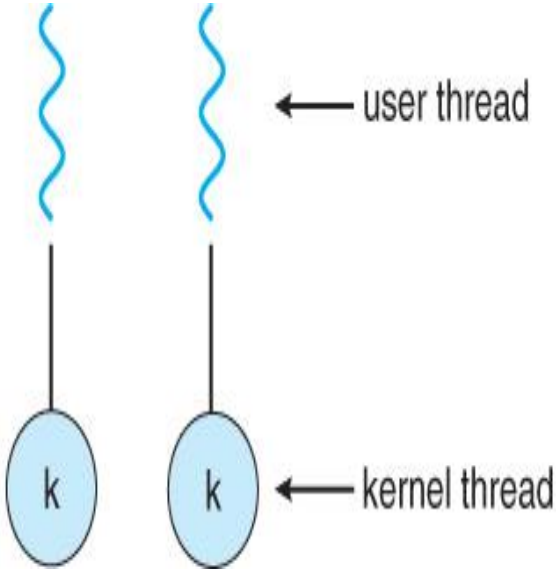
- Implementation of the many-to-one model (many user level thread to one kernel level thread) allow the application to create any number of threads that can execute concurrently. In this implementation, all threads activity is restricted to user space.

- Additionally only one thread can access the kernel at a time, so only one schedulable entity is known to the os.



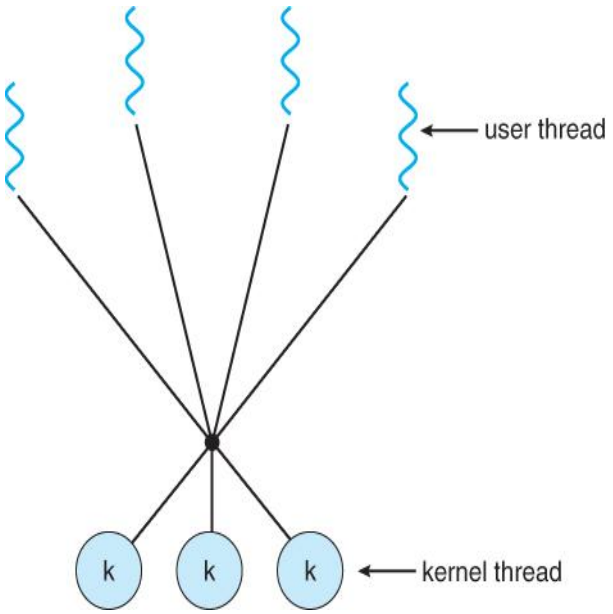
# ONE-TO-ONE MODEL

- The one-to-one model (one user level thread to one kernel level thread) is among the earliest implementations of true multithreading. In this implementation, each ULT created by the application is known to the kernel,and all threads ca access the kernel at the same time.



# MANY-TO-MANY MODEL

- The many-to-many model (many user level thread to many kernel level thread) avoids many of the limitations of the one-to-one model, while extending multithreading capabilities even further. The many-to-many model also called two-level model, minimize programing effort while reducing the cost and weight of each thread.



32

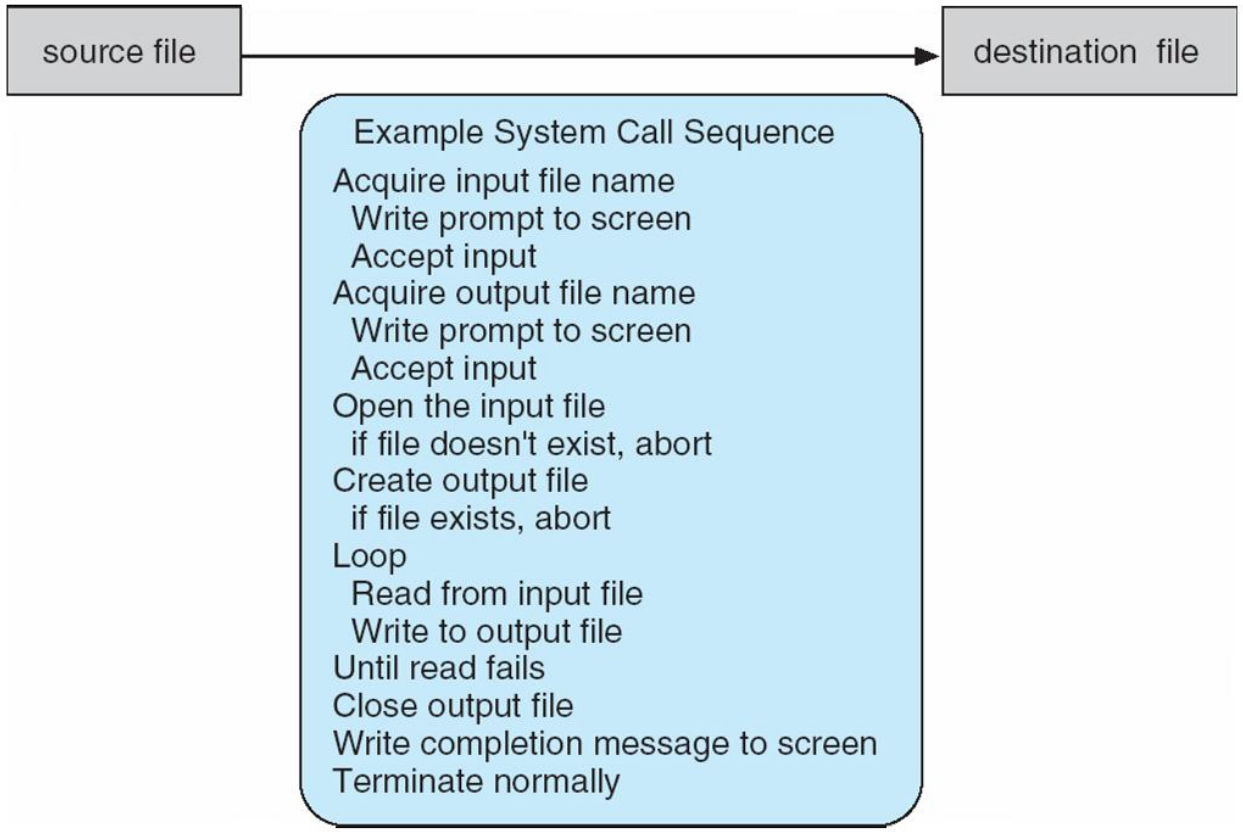
# SYSTEM CALLS

# SYSTEM CALLS

- Programming interface to the services provided by the OS.
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level Application Programming Interface (API) rather than direct system call use.
- The API specifies a set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect.

# EXAMPLE OF SYSTEM CALLS

- System call sequence to copy the contents of one file to another file



# EXAMPLE OF STANDARD API

*EXAMPLE OF STANDARD API*

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

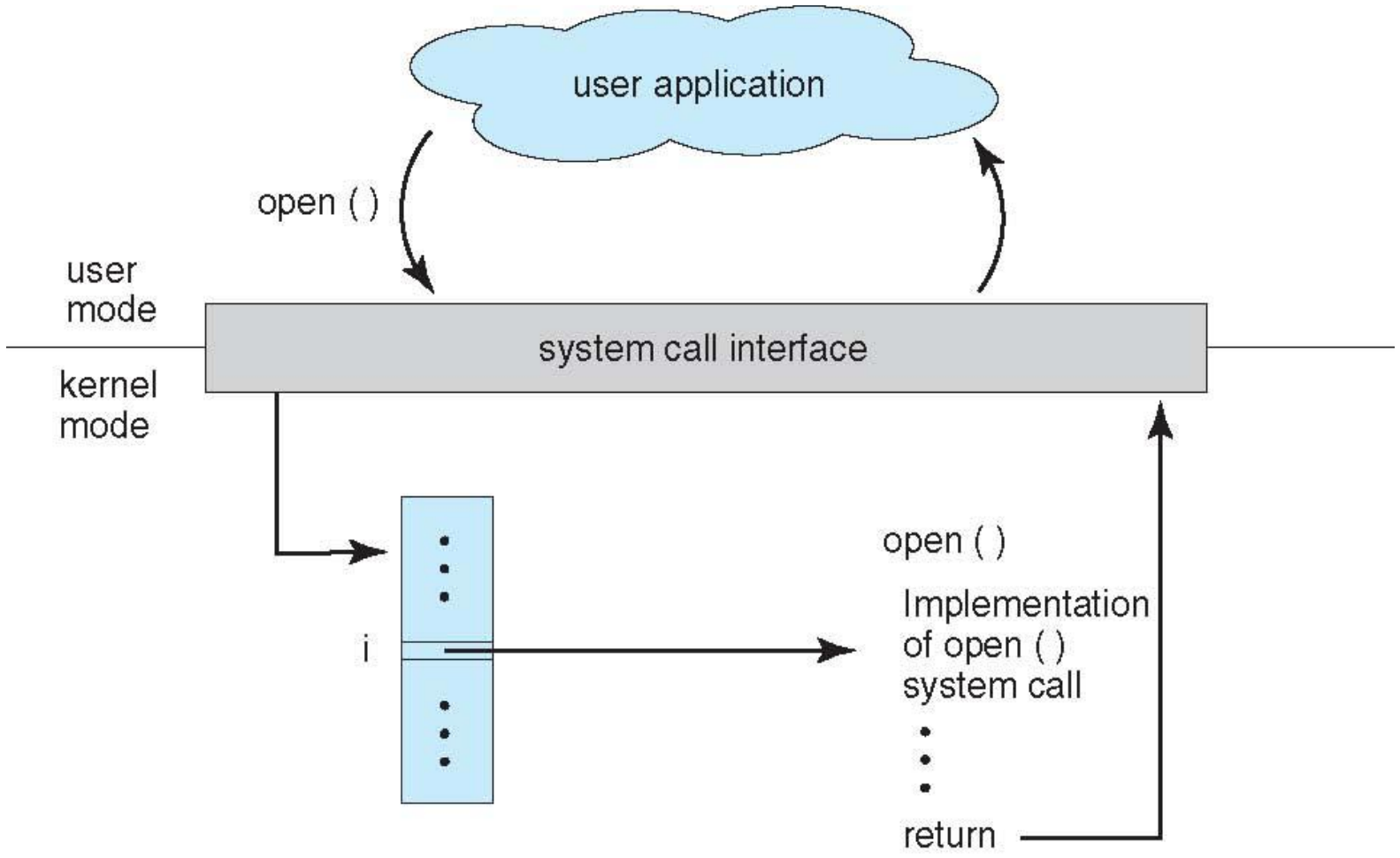
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.

# SYSTEM CALL IMPLEMENTATION

- Typically, a number associated with each system call
  - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

# API – SYSTEM CALL – OS RELATIONSHIP



# TYPES OF SYSTEM CALLS

- **Process control**
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - Debugger for determining bugs, single step execution
  - Locks for managing access to shared data between processes

# TYPES OF SYSTEM CALLS

- **File management**
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- **Device management**
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

# TYPES OF SYSTEM CALLS (CONT.)

- **Information maintenance**
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- **Communications**
  - create, delete communication connection
  - send, receive messages if message passing model to host name or process name
    - From client to server
  - Shared-memory model create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

# TYPES OF SYSTEM CALLS (CONT.)

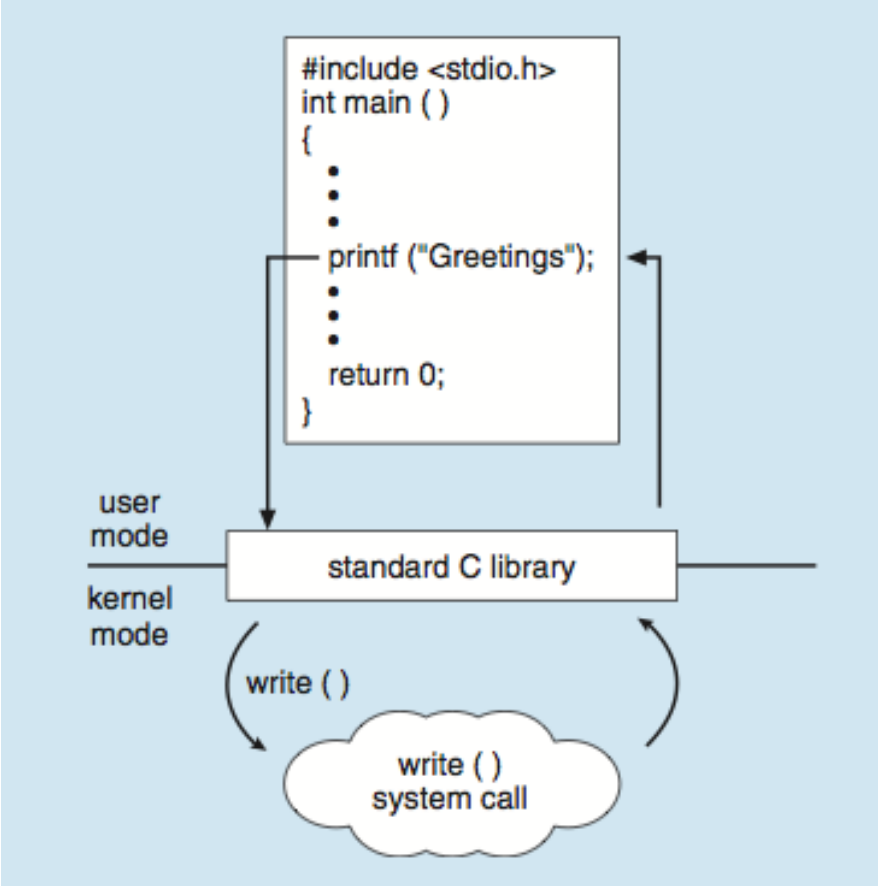
- **Protection**
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

# EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

| | Windows | Unix |
|---|---|---|
| Process Control | CreateProcess() | fork() |
| | ExitProcess() | exit() |
| | WaitForSingleObject() | wait() |
| File Manipulation | CreateFile() | open() |
| | ReadFile() | read() |
| | WriteFile() | write() |
| | CloseHandle() | close() |
| Device Manipulation | SetConsoleMode() | ioctl() |
| | ReadConsole() | read() |
| | WriteConsole() | write() |
| Information Maintenance | GetCurrentProcessID() | getpid() |
| | SetTimer() | alarm() |
| | Sleep() | sleep() |
| Communication | CreatePipe() | pipe() |
| | CreateFileMapping() | shmget() |
| | MapViewOfFile() | mmap() |
| Protection | SetFileSecurity() | chmod() |
| | InitlializeSecurityDescriptor() | umask() |
| | SetSecurityDescriptorGroup() | chown() |

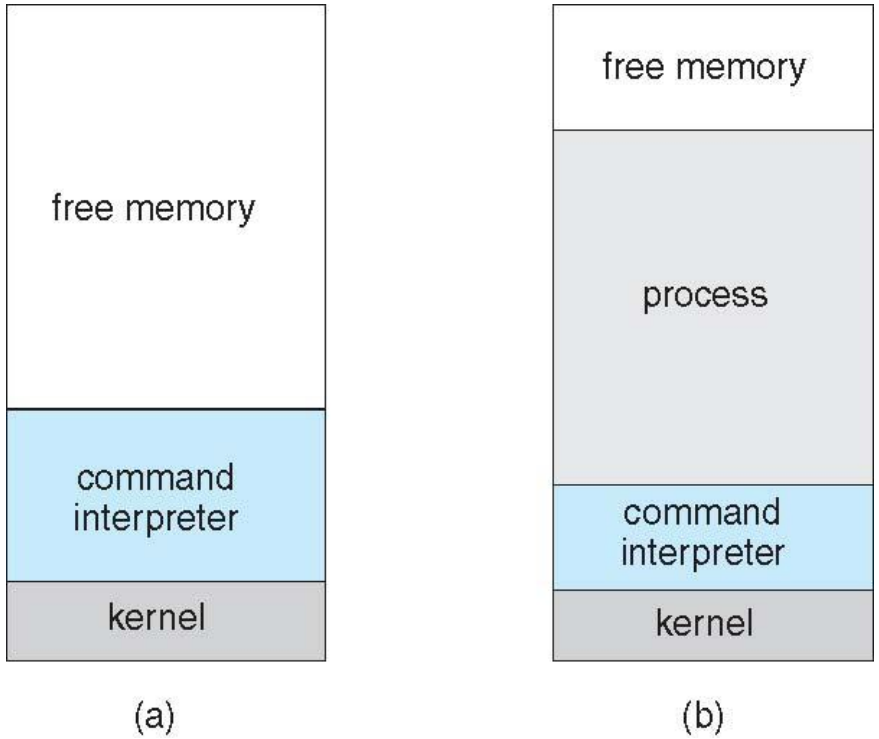
# STANDARD C LIBRARY EXAMPLE

- C program invoking printf() library call, which calls write() system call



# EXAMPLE: MS-DOS

- Single-tasking
- Shell invoked when system booted
- Simple method to run program
  - No process created
- Single memory space
- Loads program into memory, overwriting all but the kernel
- Program exit -> shell reloaded




At system startup running a program

# SYSTEM PROGRAMS

- System programs provide a convenient environment for program development and execution. They can be divided into:
  - **File manipulation**
  - **Status information sometimes stored in a File modification**
  - **Programming language support**
  - **Program loading and execution**
  - **Communications**
  - **Background services**
  - **Application programs**
- Most users' view of the operation system is defined by system programs, not the actual system calls

# SYSTEM PROGRAMS

- Provide a convenient **environment for program development and execution**
  - Some of them are simply user interfaces to system calls; others are considerably more complex

- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories

- Status information
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement  a registry - used to store and retrieve configuration information

# SYSTEM PROGRAMS (CONT.)

- **File modification**
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution-** Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

# SYSTEM PROGRAMS (CONT.)

- **Background Services**
  - Launch at boot time
    - Some for system startup, then terminate
    - Some from system boot to shutdown
  - Provide facilities like disk checking, process scheduling, error logging, printing
  - Run in user context not kernel context
  - Known as services, subsystems, daemons

- **Application programs**
  - Don't pertain to system
  - Run by users
  - Not typically considered part of OS
  - Launched by command line, mouse click, finger poke

# OPERATING SYSTEM DESIGN AND IMPLEMENTATION

- Design and Implementation of OS not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems  can vary widely
- Start the design by defining goals and specifications
- Affected by choice of hardware, type of system
- **User goals and System goals**
  - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
  - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

# IMPLEMENTATION

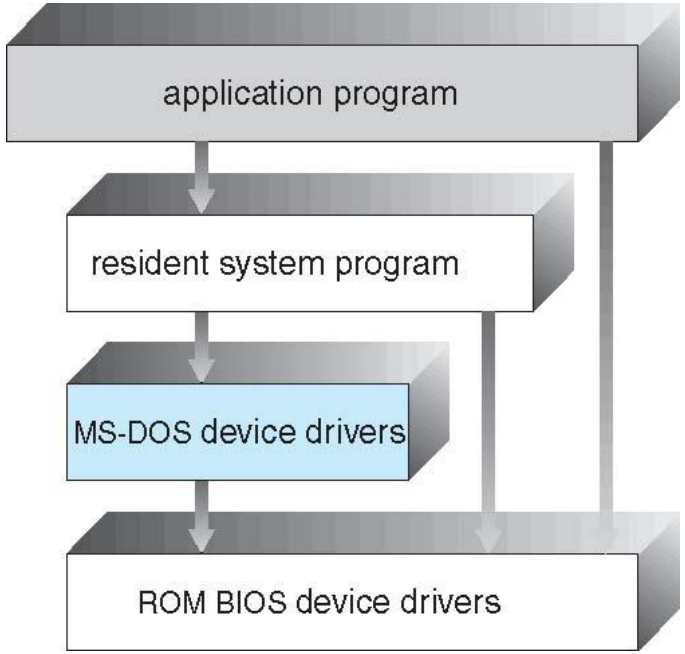
- Much variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Actually usually a mix of languages
  - Lowest levels in assembly
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to port to other hardware
  - But slower
- Emulation can allow an OS to run on non-native hardware

# OPERATING SYSTEM STRUCTURE

- General-purpose OS is very large program
- Various ways to structure ones
  - Simple structure – MS-DOS
  - More complex -- UNIX
  - Layered – an abstrcation
  - Microkernel -Mach

# SIMPLE STRUCTURE -- MS-DOS

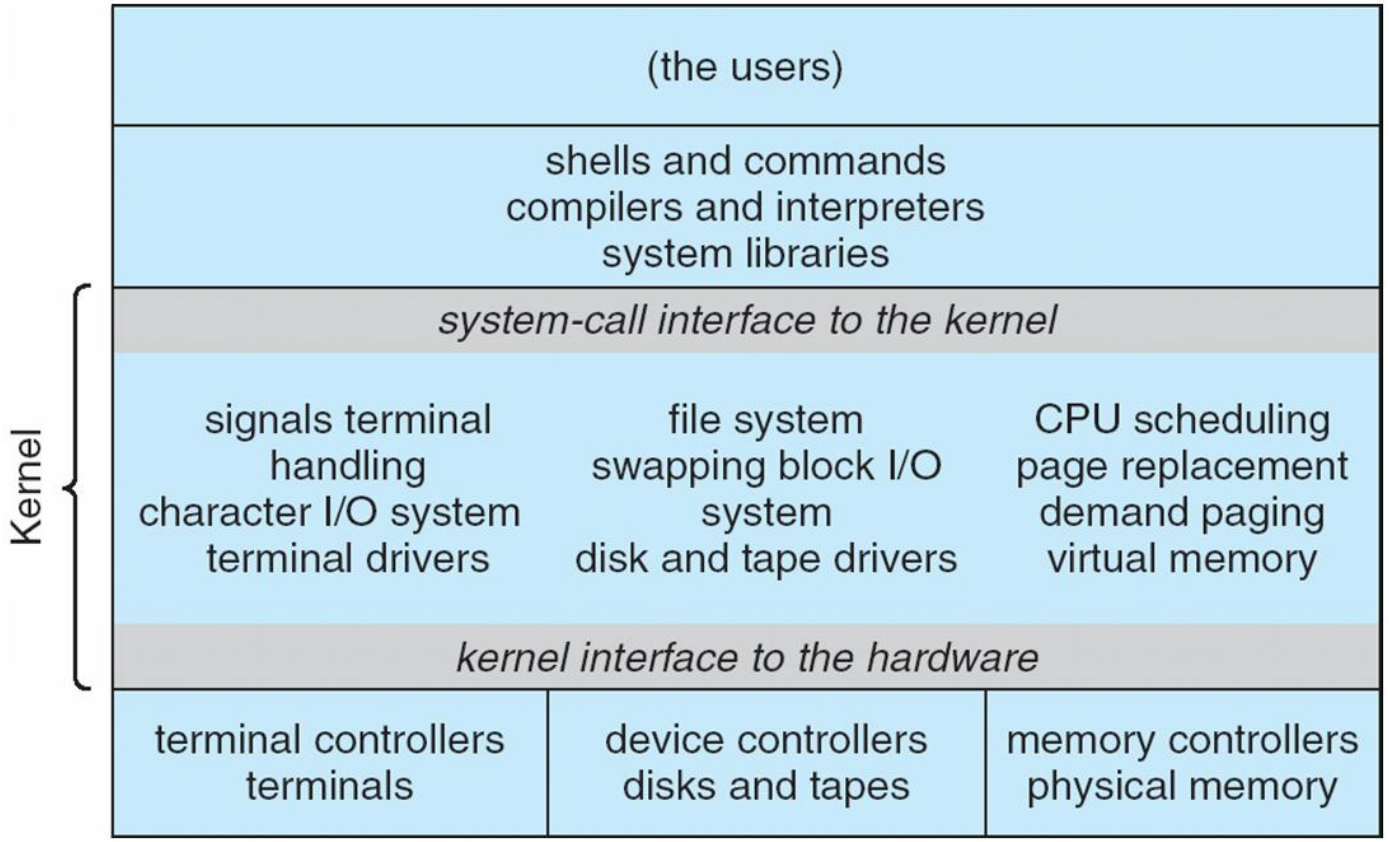
- MS-DOS – written to provide the most functionality in the least space
  - Not divided into modules
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated



# NON SIMPLE STRUCTURE - UNIX

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts
  - Systems programs
  - The kernel
    - Consists of everything below the system-call interface and above the physical hardware
    - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

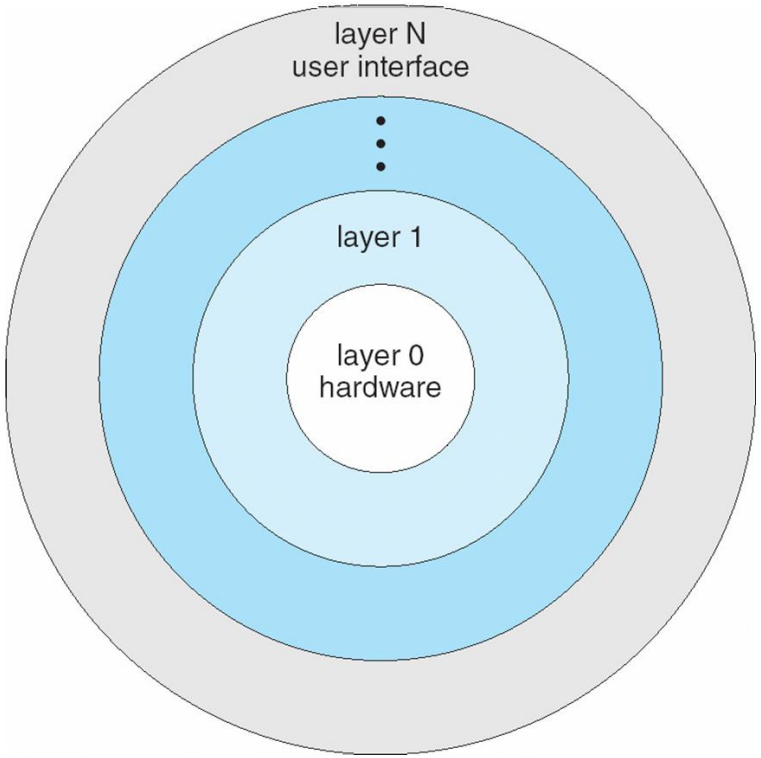
# TRADITIONAL UNIX SYSTEM STRUCTURE




Beyond simple but not fully layered

# LAYERED APPROACH

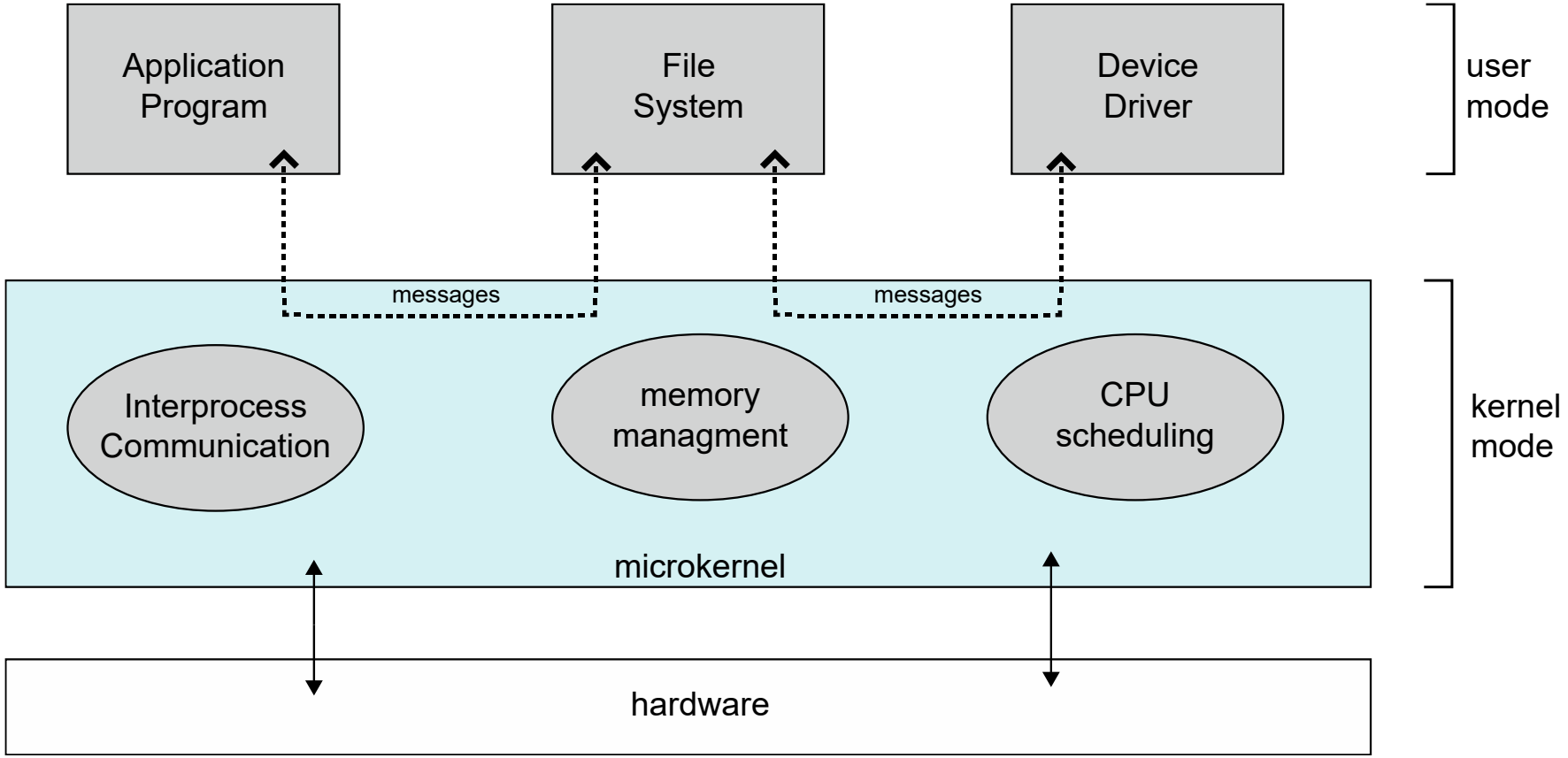
- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers



# MICROKERNEL SYSTEM STRUCTURE

- Moves as much from the kernel into user space
- Mach example of microkernel
  - Mac OS X kernel (Darwin) partly based on Mach
- Communication takes place between user modules using message passing
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication

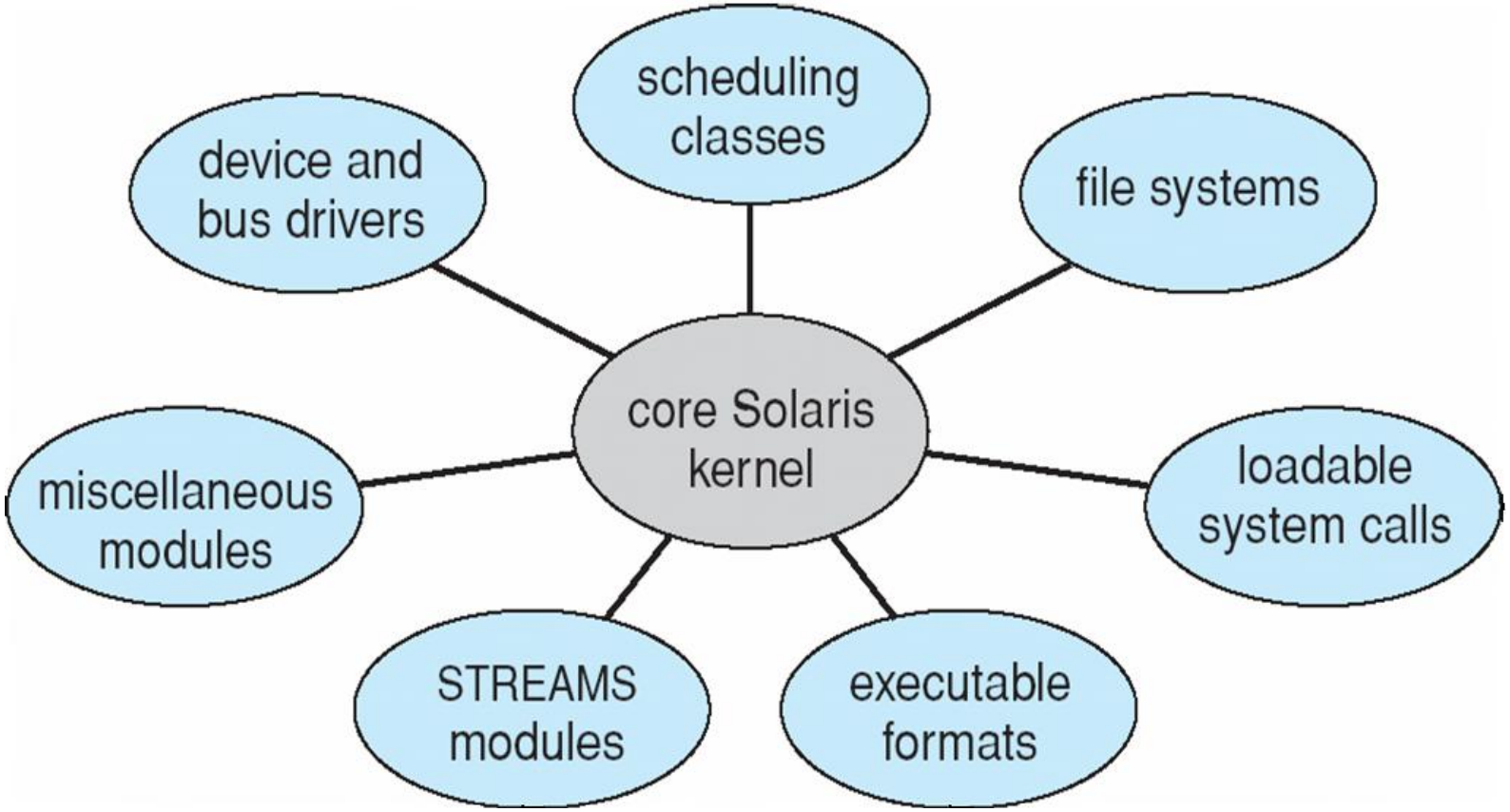
# MICROKERNEL SYSTEM STRUCTURE



# MODULES

- Many modern operating systems implement loadable kernel modules
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
  - Linux, Solaris, etc

# SOLARIS MODULAR APPROACH



# OPERATING-SYSTEM DEBUGGING

- Debugging is finding and fixing errors, or bugs
- OS generate log files containing error information
- Failure of an application can generate core dump file capturing memory of the process
- Operating system failure can generate crash dump file containing kernel memory
- Beyond crashes, performance tuning can optimize system performance
  - Sometimes using trace listings of activities, recorded for analysis
  - Profiling is periodic sampling of instruction pointer to look for statistical trends
- Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

# SYSTEM BOOT

- When power initialized on system, execution starts at a fixed memory location
  - Firmware ROM used to hold initial boot code
- Operating system must be made available to hardware so hardware can start it
  - Small piece of code – bootstrap loader, stored in ROM or EEPROM locates the kernel, loads it into memory, and starts it
  - Sometimes two-step process where boot block at fixed location loaded by ROM code, which loads bootstrap loader from disk
- Common bootstrap loader, GRUB, allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then running

# THANK YOU

Any QUESTIONS?